

Khata & Bill Management App

A complete, step-by-step blueprint for an AI coding agent to build a mobile ledger app for shopkeepers — customers, cash given/taken (udhaar) entries, itemized bills, running balances, and reports.

Platform: Android & iOS (mobile) • Audience: AI coding tool (Claude Code / Cursor) • Language: English UI with English/Urdu toggle

HOW TO USE THIS DOCUMENT

Read top to bottom once, then build phase by phase (Section 10). Each phase links to the user stories it satisfies and ends with a ready-to-paste AI prompt. Do not skip the data model (Section 6) — every screen and story depends on it. Treat the acceptance criteria as the definition of "done" for each story.

1. Product summary

Shopkeepers, wholesalers, and small traders track credit ("udhaar" / "khata") in paper notebooks. This app replaces the notebook: a **customer** is a person you deal with, and inside each customer is a running list of **entries** — every time cash is given or taken. When the shopkeeper records a new sale on credit, the entry is created as a **bill**: it asks for itemized details (item, size, color, rate, amount) and auto-generates a readable description. Each entry shifts the customer's **running balance**. Balances roll up to a business-wide total of what is owed to and by the shopkeeper.

What makes this app different

- **Bill-shaped entries.** A credit sale isn't just an amount — it's a small itemized bill the shopkeeper can share on WhatsApp.
- **Garment-aware.** Items carry size and color (built for clothing traders), and the line description is generated automatically from name + size + color.
- **Trust through history.** Every edit and delete is logged, so a customer can never dispute what changed.
- **Offline-first.** The notebook always worked without internet; so does this.

2. Target users & personas

PRIMARY – SHOP OWNER

Runs a garment / general store. Semi-literate in English, comfortable in Urdu. Wants to open the app, find a customer fast, add what they took on credit, and see the balance. One hand, on a phone, mid-conversation.

SECONDARY – SHOP HELPER

Trusted employee who records entries but should not delete customers or export data. Needs a limited role behind the same PIN-protected app.

3. Core concepts & glossary

TERM	MEANING IN THIS APP
Khata	The whole ledger for one customer – their profile plus every entry.
Customer	A person the shop deals with. Has a name, phone, optional photo, and a running balance.
Entry	One movement of money for a customer, at a point in time. Either cash out (you gave / they took on credit → they owe you more) or cash in (they paid → they owe you less).
Bill	An entry that carries itemized line details. A credit sale is entered as a bill; a plain repayment is a simple cash-in entry.
Line item	One row on a bill: item name, size, color, rate, quantity, amount, and an auto-generated description.
Running balance	The signed sum of all a customer's entries. Positive = customer owes the shop; negative = shop owes the customer.
Udhaar	Credit / borrowing. A cash-out entry is udhaar given.

THE ONE RULE THAT GOVERNS THE WHOLE APP

$\text{customer.balance} = \Sigma(\text{cash_out amounts}) - \Sigma(\text{cash_in amounts})$. The balance is **never stored as a truth you edit directly** – it is always recomputed from entries. This guarantees the ledger and the balance can never disagree.

4. Feature scope

IN SCOPE (V1)

- Customer list & profiles (add, edit, search, archive)
- Cash given / taken (udhaar) entries per customer
- Bill-style entry with itemized details & auto-generated line descriptions
- Edit / delete entries with a full change history (audit log)
- Running balance per customer and a business-wide total
- Share a bill / statement via WhatsApp & SMS
- Reports & export (PDF statement, Excel/CSV)
- Multi-language UI (English / Urdu) with RTL support for Urdu
- App lock: PIN / biometric login

OUT OF SCOPE (V1) – NOTE FOR LATER

- Cloud sync & multi-device (v2) – design the data layer so it can be added
- Online payments / payment gateway
- Inventory / stock counts (bills reference free-text items, not a stock table)

5. Recommended tech stack

Chosen for offline-first mobile, a single codebase for Android & iOS, and a data layer that can later sync to the cloud.

CONCERN	RECOMMENDATION
Framework	React Native + Expo (TypeScript). One codebase, fast iteration, easy builds.
Local database	WatermelonDB or SQLite (expo-sqlite / Drizzle ORM) – relational, offline-first, sync-ready.
State / data access	Repository layer over the DB; TanStack Query for reactive reads.
Navigation	Expo Router (file-based) or React Navigation (stack + tabs).
Internationalization	<code>i18next</code> + <code>react-i18next</code> ; <code>I18nManager</code> for Urdu RTL.
App lock	<code>expo-local-authentication</code> (biometrics)+ a hashed PIN in <code>expo-secure-store</code> .
PDF / share	<code>expo-print</code> (bill → PDF), <code>expo-sharing</code> / deep links for WhatsApp & SMS.
Export	CSV writer + <code>xlsx</code> for Excel; save via <code>expo-file-system</code> .

If the team already standardizes on Flutter, the same data model and stories apply – swap RN packages for Drift (SQLite), `go_router`, `easy_localization`, `local_auth`, and printing.

6. Data model

Five tables. All primary keys are UUID strings. All money is stored as **integer paisa/cents** (never floats) to avoid rounding errors. All timestamps are UTC ISO strings.

customers

FIELD	TYPE	NOTES
id	uuid	PK
name	text	Required
phone	text	For WhatsApp/SMS; optional
photo_uri	text	Optional local image
address	text	Optional
opening_balance	int	Paisa; balance carried in at customer creation
is_archived	bool	Soft-hide without deleting history
created_at / updated_at	timestamp	—

entries

FIELD	TYPE	NOTES
id	uuid	PK
customer_id	uuid	FK → customers
direction	enum	<code>cash_out</code> (credit given) or <code>cash_in</code> (payment received)
type	enum	<code>bill</code> (has line items) or <code>simple</code> (amount only)
entry_date	date	Business date, editable (defaults to today)
amount	int	Paisa; for bills = sum of line item amounts (derived, stored for speed)
note	text	Free text
attachment_uri	text	Optional photo of a paper receipt
is_deleted	bool	Soft delete — kept for history
created_at / updated_at	timestamp	—

line_items

FIELD	TYPE	NOTES
id	uuid	PK
entry_id	uuid	FK → entries
item_name	text	e.g. "Lawn Kurta"
size	text	S / M / L / XL / XXL / custom
color	text	From the garment color palette (see 7.4)
quantity	int	Default 1
rate	int	Paisa, per unit
amount	int	= quantity × rate
description	text	Auto-generated from name + size + color (see 7.3); editable override allowed

audit_log

FIELD	TYPE	NOTES
id	uuid	PK
entity	enum	customer / entry / line_item
entity_id	uuid	The affected record
action	enum	create / edit / delete
diff	json	Old → new values for changed fields
actor	text	owner / helper
created_at	timestamp	—

settings (single row)

Business name, logo, currency symbol (default Rs/PKR), language (en / ur), PIN hash, biometric-enabled flag, default bill footer text.

7. The bill entry – the core interaction

This is the feature the whole app is built around, so it is specified in full.

7.1 When a bill is created

From a customer's khata, the shopkeeper taps **+ New Entry** and chooses the direction. Choosing **"Gave on credit (cash out)"** and toggling **"Itemized bill"** opens the bill form. A plain "gave cash" or "received payment" uses the simple form (amount + note only).

7.2 Bill form – fields asked

FIELD	INPUT	BEHAVIOUR
Date	Date picker	Defaults to today; editable
Item name	Text + recent suggestions	Required per line
Size	Chips / dropdown	S, M, L, XL, XXL, Free size, custom
Color	Swatch picker	Garment palette (7.4)
Quantity	Stepper	Default 1
Rate	Number pad	Per-unit price
Amount	Auto (qty × rate)	Read-only, editable override
Description	Auto-generated	Read-only by default, tap to edit (7.3)
+ Add line	Button	Repeat for multiple items
Note	Text	Bill-level note, optional
Attachment	Camera / gallery	Optional photo of paper slip

A live **bill total** updates as lines are added. Saving creates one `entry` (type = bill) with its `line_items`, updates the running balance, and writes a `create` row to the audit log.

7.3 Auto-generated description rule

Format: {quantity}× {color} {item_name} ({size})

Examples: 1× Navy Blue Lawn Kurta (M) · 3× Maroon Cotton Shalwar (XL) . Empty parts are dropped gracefully: no size → 2× White Kameez . Regenerate whenever name, size, color, or quantity changes, *unless* the user has manually edited the description (set a `description_touched` flag to stop overwriting their text).

7.4 Garment color palette

A fixed swatch set keeps colors consistent and searchable. Store the label; render the swatch from the hex.

COLOR	HEX	COLOR	HEX
White	#F7F7F5	Black	#1E1E1E
Navy Blue	#22335B	Sky Blue	#8FB8DE
Maroon	#7B2D33	Red	#C0392B
Bottle Green	#20603D	Mustard	#D4A431
Beige	#D9C9A8	Grey	#8A8D91
Pink	#E4A7B8	Purple	#6E4A8E
Brown	#6B4A2F	Off-White / Cream	#EFE7D3

8. User stories (step by step)

Grouped by epic. Format: **As a** [role], **I want** [goal], **so that** [reason]. Each carries acceptance criteria (AC).

Epic A – App lock & onboarding

A1. As a shop owner, I want to set up my business name, currency, and language on first launch, so the app speaks my language and shows my shop.

AC: First run shows a 3-step setup (business → language → set PIN). Language switch flips the whole UI, Urdu renders RTL. Settings persist.

A2. As a shop owner, I want the app locked behind a PIN or fingerprint, so no one else can read my khata.

AC: App requires PIN/biometric on launch and after 2 min in background. PIN is hashed, never stored plain. 5 wrong tries → 30s lockout.

Epic B – Customers

B1. As a shop owner, I want to add a customer with name, phone, and optional photo, so I can start their khata.

AC: Name required. Phone validated. Optional opening balance. Saving returns to the list with the new customer on top.

B2. As a shop owner, I want a searchable customer list showing each balance, so I find anyone in seconds and see who owes me.

AC: Live search by name/phone. Each row shows name, balance (red = owes me, green = I owe), last-activity date. Sort by balance or recency.

B3. As a shop owner, I want to edit or archive a customer, so I can fix details or hide inactive ones without losing history.

AC: Edit logs to audit. Archive hides from the main list but keeps all entries; archived customers viewable under a filter.

Epic C – Entries & bills

C1. As a shop owner, I want to open a customer and see their full entry list with the running balance, so I know exactly where we stand.

AC: Khata screen lists entries newest-first with date, description/note, cash-in vs cash-out amount, and a big running balance header. Deleted entries hidden but reachable via history.

C2. As a shop owner, I want to record a simple cash-in (payment received) or cash-out, so quick transactions take two taps.

AC: Direction toggle, amount via number pad, date (default today), optional note. Save updates balance instantly and logs to audit.

C3. As a shop owner, I want to record a credit sale as an itemized bill (item, size, color, rate, qty), so the customer sees exactly what they bought.

AC: Full bill form (7.2). Multiple line items. Each line auto-generates its description (7.3). Live bill total. Saving creates entry + line items, updates balance, logs audit.

C4. As a shop owner, I want to edit or delete any entry, so I can correct mistakes – but every change must be recorded.

AC: Edit reopens the same form; save writes an `edit` audit row with old→new diff. Delete is a soft delete (`is_deleted`) with an `delete` audit row; balance recomputes.

C5. As a shop owner, I want to view the change history of an entry, so I can prove nothing was tampered with.

AC: "History" on an entry shows a timeline from the audit log: who, when, what changed.

Epic D – Sharing, reports & export

D1. As a shop owner, I want to send a bill or full statement to a customer on WhatsApp/SMS, so they have a record without paper.

AC: "Share" builds a formatted text (business name, items, total, balance) and opens WhatsApp deep link to their number, with SMS fallback. Also "Share as PDF".

D2. As a shop owner, I want a dashboard of total receivable, total payable, and today's activity, so I know my day at a glance.

AC: Home shows total owed to me, total I owe, count of customers, today's cash in/out. Tapping a stat filters the list.

D3. As a shop owner, I want to export a customer statement as PDF and all data as Excel/CSV, so I can keep records and share with an accountant.

AC: PDF statement per customer (branded, date-ranged). CSV/Excel of all customers & entries. Files saved and shareable.

9. Screen-by-screen specification

SCREEN	CONTENTS & KEY ACTIONS
Lock screen	PIN pad + biometric prompt. Appears on launch / resume.
Home / Dashboard	Business header, receivable/payable totals, today's activity, quick "+ New Customer". Bottom tabs: Home · Customers · Reports · Settings.
Customer list	Search bar, sort control, rows (name · balance · last activity), FAB "+".
Add / edit customer	Name, phone, photo, address, opening balance. Save / cancel.
Customer khata (detail)	Big balance header (color-coded), entry list newest-first, "+ New Entry", "Share statement". Each entry row → detail / edit.
New / edit entry – simple	Direction toggle, amount pad, date, note. Save.
New / edit entry – bill	Line-item editor (7.2), live total, note, attachment, auto descriptions. Save.
Entry detail	Full bill view, "Share", "Edit", "Delete", "History".
Entry history	Audit timeline for that entry.
Reports	Totals, date-range filter, export to PDF / Excel / CSV.
Settings	Business profile, language (EN/UR), currency, change PIN, biometric toggle, bill footer, backup/restore.

10. Build roadmap – phase by phase

Build in this order. Each phase is independently runnable and testable before moving on.

PHASE 0 – FOUNDATION

Init Expo + TypeScript project, navigation shell (tabs + stacks), theme tokens, i18next with English + Urdu resource files and RTL flip. Set up the SQLite/WatermelonDB schema for all five tables and a repository layer.
Covers no stories yet – this is the skeleton.

PHASE 1 – CUSTOMERS

Customer list, search, add/edit, archive, and the empty balance display. **Stories B1–B3.** Balance computed from (still empty) entries.

PHASE 2 – SIMPLE ENTRIES & RUNNING BALANCE

Customer khata screen, simple cash-in/cash-out entries, running balance math, home dashboard totals.

Stories C1, C2, D2.

PHASE 3 – ITEMIZED BILLS

The bill form, line items, size/color pickers, garment palette, and the auto-description engine. **Story C3.** This is the centerpiece – budget the most time here.

PHASE 4 – EDIT, DELETE & HISTORY

Editing and soft-deleting entries with the audit log wired through every mutation, plus the history timeline.

Stories C4, C5.

PHASE 5 – SHARING & EXPORT

WhatsApp/SMS share, PDF bill & statement, Excel/CSV export. **Stories D1, D3.**

PHASE 6 – SECURITY & ONBOARDING

PIN + biometric lock, first-run setup wizard, settings screen, local backup/restore. **Stories A1, A2.**

PHASE 7 – POLISH & RELEASE

Empty states, error handling, large-number formatting, accessibility, app icons/splash, store builds (EAS). QA against every AC in Section 8.

11. Ready-to-paste AI prompts

Paste these into the coding agent one phase at a time. Give it this whole document as context first.

PHASE 0 – FOUNDATION

Set up a new Expo + TypeScript app called "Khata". Add React Navigation with bottom tabs (Home, Customers, Reports, Settings) and stack navigators inside each. Configure i18next with en and ur locale files and enable RTL when ur is active. Create a SQLite data layer (expo-sqlite + Drizzle) with tables: customers, entries, line_items, audit_log, settings exactly as defined in the data model. Expose a typed repository module (createCustomer, listCustomers, createEntry, computeBalance, logAudit, ...). Store all money as integer paisa. Add seed data for 3 demo customers.

PHASE 1 – CUSTOMERS

Build the Customers feature. A searchable list showing name, running balance (red if the customer owes us, green if we owe them), and last-activity date, sortable by balance or recency, with a FAB to add. Build the Add/Edit Customer form (name required, phone, photo via image picker, address, opening balance). Support archive (soft-hide) with an archived filter. Every create/edit/archive writes an audit_log row. Balance = opening_balance + Σ cash_out – Σ cash_in, always recomputed from entries.

PHASE 2 – ENTRIES & BALANCE

Build the Customer Khata screen: a color-coded running-balance header and an entry list (newest first) showing date, note/description, and the amount as cash-in (green) or cash-out (red). Add a "New Entry" flow with a simple mode: direction toggle (gave / received), amount number pad, editable date defaulting to today, optional note. Saving updates the balance and logs audit. Build the Home dashboard: total receivable, total payable, customer count, today's cash in/out, each tappable to filter.

PHASE 3 – ITEMIZED BILLS

Extend New Entry with a "bill" mode for credit sales. A line-item editor where each row has: item name (with recent suggestions), size chips (S/M/L/XL/XXL/Free/custom), a color swatch picker using the fixed garment palette, quantity stepper, per-unit rate, and an auto-computed amount ($\text{qty} \times \text{rate}$). Auto-generate each line's description as "{qty}× {color} {item} ({size})", dropping empty parts, and stop overwriting it once the user edits it manually. Show a live bill total. Support multiple lines, a bill-level note, and an optional attachment photo. Saving creates one entry (type=bill) with its line_items and updates the balance.

PHASE 4 – EDIT, DELETE & HISTORY

Allow editing any entry (reopen the same simple/bill form) and soft-deleting it (is_deleted=true). Every edit writes an audit_log row with a field-level old→new diff; every delete writes a delete row. Recompute the balance after any mutation. Add an Entry History screen that renders the audit timeline for an entry (who, when, what changed). Deleted entries are hidden from the khata but reachable through history.

PHASE 5 – SHARING & EXPORT

Add sharing: build a formatted bill/statement text (business name, line items, total, current balance) and open a WhatsApp deep link to the customer's phone, with an SMS fallback. Add "Share as PDF" using expo-print for both a single bill and a full customer statement (branded header, date range). Add a Reports screen that exports all customers and entries to CSV and Excel (xlsx), saved via expo-file-system and shareable.

PHASE 6 – SECURITY & ONBOARDING

Add an app lock: a hashed PIN in secure-store plus biometric unlock (expo-local-authentication), required on launch and after 2 minutes in background, with a 30-second lockout after 5 failed attempts. Build a first-run setup wizard (business name/logo → language → set PIN) and a Settings screen (business profile, language EN/UR, currency, change PIN, biometric toggle, bill footer text, local backup/restore).

12. Definition of done

The app is v1-complete when every user story in Section 8 passes its acceptance criteria, and:

- Balances always equal the recomputed sum of entries — never drift.
- Every create/edit/delete on a customer or entry appears in the audit log.
- The app opens, adds a customer, records a bill, and shares it — fully offline.
- Switching to Urdu flips the entire UI to RTL with translated strings.
- Money never shows floating-point rounding errors.
- Nothing is reachable without passing the PIN/biometric lock.

End of specification · Khata & Bill Management App · v1.0